

DESCRIPTION OF THE OBJECT OF COPYRIGHT

(for depositing computer programs and technical documentation)

1. Title of the copyright object:

SafeNest computer program – microclimate monitoring and carbon monoxide alert

2. Author:

Berdybek ErasyI Erkebulanuly.

3. Creation Date:

September 19, 2026

4. Purpose and scope of application:

The system is designed for continuous monitoring of carbon monoxide (CO) levels, temperature, and humidity in residential, utility, and technical spaces. The software provides data collection, mathematical processing, and local display, as well as remote multilingual user alerts about emergency situations, including automatic room photography and push notifications.

5. Functional capabilities of the program:

- **Telemetry collection:** Continuous asynchronous polling of analog and digital sensors (MQ-9, DHT22) at a specified frequency.
- **Mathematical analysis:** Calculates the estimated CO concentration in ppm and the Heat Index, and compares the calculated CO value with the adjustable alarm threshold
- **FreeRTOS Multithreading:** Dual-core task sharing - generating real-time audio alarms in an isolated thread on Core 0, and processing network protocols and collecting data on Core 1.
- **Local indication:** Control of a passive piezoelectric emitter and a character LCD display

via the I2C bus with a cyclic scrolling mode for telemetry screens.

- **Cross-controller cloud sync:** Pass alarm triggers and control commands between controllers via the Firebase Realtime Database.
- **Emergency alerts and photo capture:** The camera module is initialized when the ESP32-CAM starts. Upon receiving a cloud command via the Firebase Realtime Database, a JPEG image is captured and sent via the Telegram Bot API.

6. Hardware requirements (Hardware configuration):

- Main computing module: ESP32 DevKit V1.
- Communication and photo recording module: ESP32-CAM AI-Thinker.
- Sensors: Combustible gas and carbon monoxide sensor MQ-9, temperature and humidity sensor DHT22.
- Peripherals: 16×2 character LCD display (with PCF8574 I2C adapter), passive piezo buzzer.

7. Software requirements and development environment:

- **Development environment:** Arduino IDE / Framework ESP-IDF
- **Dependencies and libraries used:** `WiFi.h` , `WiFiClientSecure.h` , `HTTPClient.h` , `esp_camera.h` , `FirebaseESP32.h` , `LiquidCrystal_I2C.h` , `DHT.h` , `Wire.h` , `Preferences.h` , `soc/soc.h` , `soc/rtc_cntl_reg.h`
- Telegram uses HTTPS when sending text messages and images; server certificate verification is disabled in both software modules.

8. Programming language:

C/C++

9. Source code size:

Video recording module (ESP32-CAM): 5KB/135 lines of code

Central control module (ESP32): 22KB/509 lines of code

10. Program operation algorithm and program structure

The complex's operating algorithm consists of two independent software modules synchronized via the Firebase Realtime Database cloud database .

10.1. Main Control Module Algorithm (ESP32 DevKit V1)

1. System initialization (setup() function):
 - Setting up input/output ports (BUZZER_PIN piezo buzzer).
 - Initialization of the I2C bus and display (Wire.begin(), lcd.init()).
 - Loading the saved CO threshold from the NVS non-volatile memory (prefs.begin("safeness")).
 - Initialize the DHT22 sensor (dht.begin()), connect to Wi-Fi and authorize in Firebase (Firebase.begin()).
 - Starting FreeRTOS multithreading: creating and pinning a standalone siren task sirenTask to Core 0 (xTaskCreatePinnedToCore()).
2. Data collection and initial filtering (Main loop()):
 - Reading temperature and humidity readings via dht.readTemperature() and dht.readHumidity() every 2 seconds.
 - Calculating the heat index using a mathematical formula in the calculateHeatIndex() function.
 - Polling the MQ-9 analog input with noise filtering by averaging 30 measurements in the getSmoothedAnalog() function.
 - The convertToPPM() function calculates an estimated CO concentration in ppm from the MQ-9 sensor signal using a specified formula and coefficients. The accuracy of the calculation for a specific sensor requires confirmation through calibration and testing.
3. Anxiety assessment and logic processing:
 - Temperature and humidity alarm condition check: When the temperature simultaneously exceeds 50°C and the relative humidity drops below 6%, an alarm is activated. This condition is a preset software criterion and does not, in itself, confirm the presence of a fire.
 - When the CO_PPM_THRESHOLD is exceeded, the alarmConfirmCounter counter increments. When the CO value drops below 80% of the threshold, the counter decrements if it is greater than zero; in the intermediate range, the counter remains unchanged. When the counter reaches 3, an alarm is activated.
 - Accumulation of alarmConfirmCounter confirmation counter to eliminate false alarms.
 - When the danger is confirmed, the realAlarmState = true flag is set and the alarm start time is recorded as alarmStartTime.
4. Parallel sound control (FreeRTOS sirenTask thread on Core 0):
 - The task continuously monitors the realAlarmState flag.
 - When an alarm occurs, it generates audio frequency pulses on the piezoelectric emitter using tone(BUZZER_PIN) depending on the selected sirenMode (tone, intermittent or tone-modulated siren).
5. Cloud sharing and sending alarms:

- The `pushTelemetry()` function generates a JSON packet and sends telemetry to the `/telemetry` branch of the Firebase database.
 - The `pollControlNode()` function reads control parameters from the `/control` branch (`isMuted` mute, `currentLang` language, device reset).
 - When an alarm occurs, the `sendTelegramAlarm()` function generates text in the selected language via `buildAlarmMessage()` and sends it via the Telegram Bot API, and also sets the `/control/command_snap = true` flag in Firebase to initiate snapshotting.
6. Local indication:
- The `drawCoBar()` function draws a graphical CO level bar indicator on the LCD display.
 - Cycles through telemetry screens (temperature/humidity, CO PPM, `getUptime()` and IP address).

10.2. Video recording module algorithm (ESP32-CAM)

1. Initializing the hardware (`setup()` function):
 - Disabling the `RTC_CNTL_BROWN_OUT_REG` hardware and software voltage drop detector to prevent reboots during peak Wi-Fi consumption.
 - Configuring DVP bus pins and setting the JPEG QVGA frame format in the `initCamera()` function.
 - Connect to Wi-Fi and initialize the Firebase client.
2. Polling a cloud trigger (Main loop()):
 - Reads the `/control/command_snap` flag from the Firebase database once per second.
 - Reads the target user chat ID `/control/user_tg_id`.
3. Capturing and sending a frame (`captureAndSend()` function):
 - When `command_snap == true` is received, it requests a frame from the matrix via `esp_camera_fb_get()`.
 - To transfer the image, a TLS connection is established with the `api.telegram.org` server using `WiFiClientSecure`. In the presented version, server certificate verification is disabled by calling `setInsecure()`.
 - Generates a multipart/form-data HTTP-POST request and streams the binary JPEG image data to the Telegram Bot.
 - Frees the frame buffer `esp_camera_fb_return(fb)` and resets the flag in the database: `/control/command_snap = false`.

11. Composition of deposited materials:

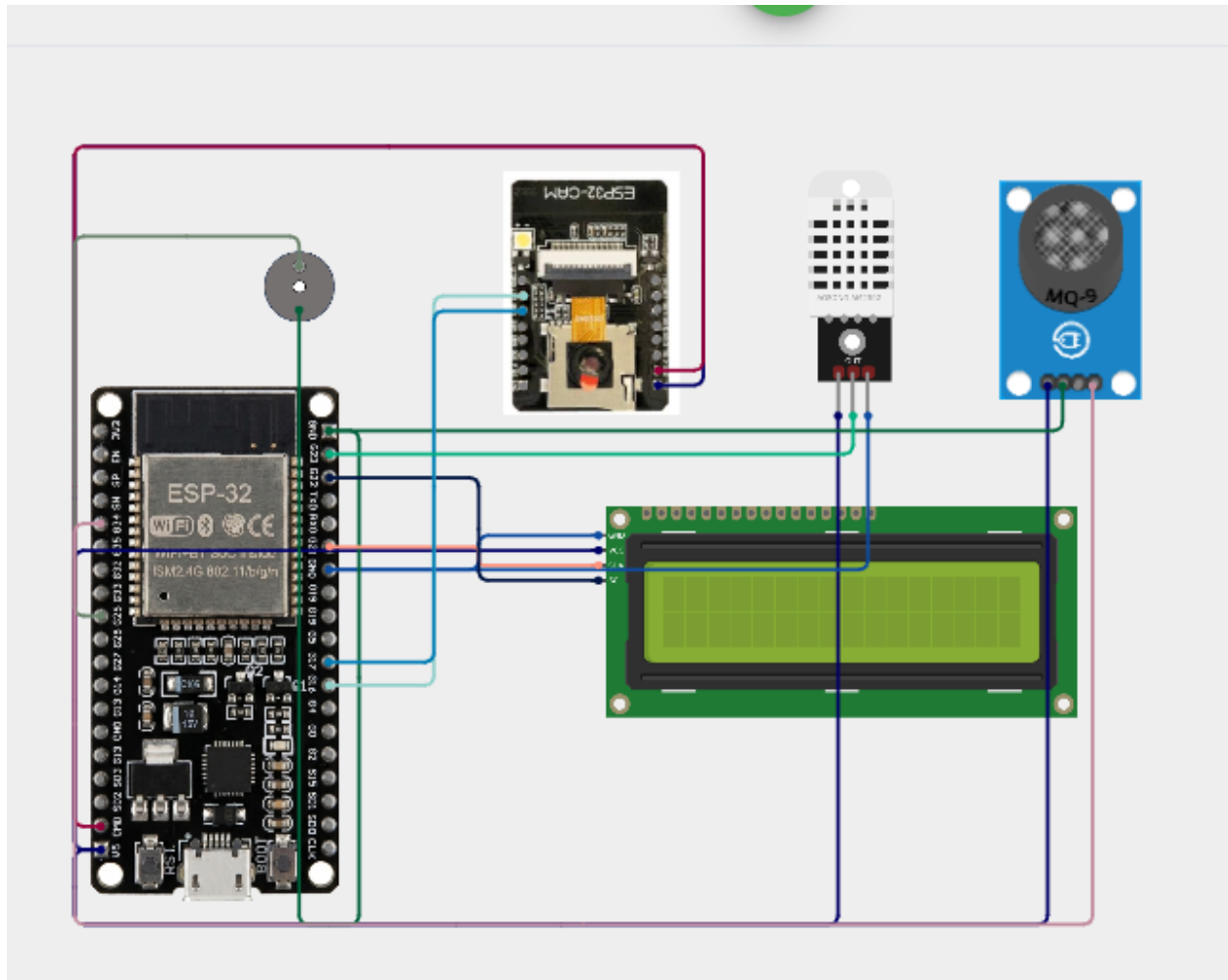
1. Text description of the copyright object
2. Listing of the source text of the computer program (provided in a separate file `LISTING_OF_SOURCE_TEXT_SAFENEST.docx`).
3. Technical documentation, graphic diagrams and 3D models of the case

APPENDICES (Deposited Materials)

APPENDIX 1. Software source code

The source code of the ESP32 DevKit V1 module and the ESP32-CAM module is located in a separate document, “LISTING OF THE SOURCE TEXT OF THE COMPUTER PROGRAM” .

APPENDIX 2. Component connection diagram



APPENDIX 3. 3D models of the device body



